



REPUBLIQUE DU SENEGAL

Un Peuple – Un But – Une Foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE

| INSTITUT SUPERIEURE EN INFORMATIQUE

GROUPE
ISI

MASTER 1 Réseaux et Système

Rapport détaillé - Régression Linéaire Multiple

Prédiction du prix d'une maison à partir de HouseSize, DistanceToCity et Location - réalisé sur Google Colab

Présenté par :

Nurja Guychel MBOUMBA

Fabrice Olivier lamine TINE

Mamadou Bamba DIOUF

Encadreur :

M. Barro

PLAN

Introduction

1. Mise en place de l'environnement de travail

1.1 Importation des bibliothèques

2. Chargement et préparation des données

2.1 Lecture et nettoyage du fichier

2.2 Encodage de la variable catégorielle « Location »

3. Construction du modèle

3.1 Séparation des variables d'entrée (X) et de sortie (y)

3.2 Division en jeu d'entraînement et jeu de test

3.3 Création et entraînement du modèle

4. Évaluation du modèle

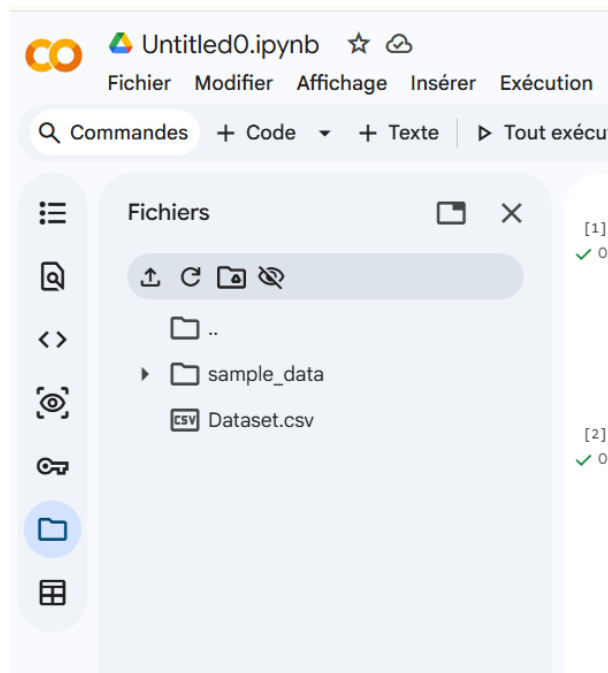
Conclusion

Introduction

La consigne demandait de mettre en place un modèle de régression linéaire multiple permettant de prédire une variable de sortie le prix d'une maison (Price) à partir de trois variables d'entrée : HouseSize (la surface), DistanceToCity (la distance au centre-ville) et Location (une variable catégorielle représentant la zone). L'ensemble du travail a été réalisé dans un notebook Google Colab.

1. Mise en place de l'environnement de travail

La première étape a consisté à créer un notebook Colab (Untitled0.ipynb) et à y importer le fichier de données Dataset.csv via le panneau « Fichiers » situé dans la barre latérale gauche (icône dossier). Une fois importé, le fichier apparaît dans la liste et devient accessible depuis le code, sous le nom Dataset.csv.

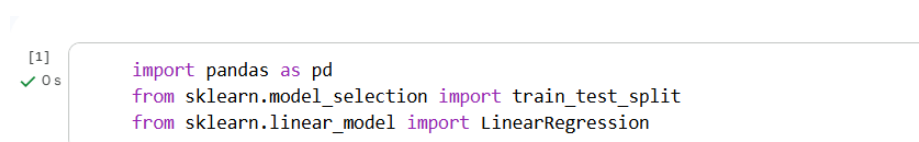


Capture 1 - Panneau « Fichiers » de Google Colab : le fichier Dataset.csv a été importé et est visible dans l'arborescence, aux côtés du dossier sample_data fourni par défaut.

Remarque importante : ce fichier est stocké temporairement sur la machine virtuelle de la session Colab. Il faut le réimporter si la session est fermée ou redémarrée.

1.1 Importation des bibliothèques

La première cellule de code importe les outils nécessaires : pandas pour manipuler les données, train_test_split pour diviser le jeu de données, et LinearRegression pour construire le modèle.



Capture 2 - Cellule [1] : import des bibliothèques pandas et scikit-learn. Exécutée en 0 seconde, sans erreur.

2. Chargement et préparation des données

2.1 Lecture et nettoyage du fichier

La cellule [2] réalise trois actions successives : lire le fichier CSV avec pd.read_csv(), supprimer les lignes contenant des valeurs manquantes avec df.dropna() (ces lignes vides pourraient faire échouer l'entraînement du modèle), puis afficher les cinq premières lignes avec df.head() pour vérifier visuellement que le chargement s'est bien passé.

Charger et nettoyer le fichier

[2]
✓ 0 s

```
# 1. Lire le fichier
df = pd.read_csv('Dataset.csv')

# 2. Supprimer les lignes vides qui bloquaient le modèle
df = df.dropna()

# 3. Voir si le fichier est bien chargé
df.head()
```

	HouseSize	DistanceToCity	Location	Price
1	48.6	27.90	B	19.35
3	65.2	25.99	A	25.37
4	47.7	29.19	A	19.66
5	47.7	32.02	C	18.61
6	65.8	39.43	B	27.20

Étapes suivantes : [Générer du code avec df](#) [New interactive sheet](#)

Capture 3 - Cellule [2] : lecture du CSV, suppression des lignes vides, puis aperçu du tableau. On y voit les quatre colonnes : HouseSize, DistanceToCity, Location et Price.

On remarque que l'index des lignes affichées commence à 1 et saute certaines valeurs (1, 3, 4, 5, 6...) : cela confirme que `dropna()` a bien supprimé des lignes contenant des valeurs manquantes (par exemple la ligne d'index 0 et 2 ont disparu du tableau).

2.2 Encodage de la variable catégorielle « Location »

Un modèle de régression linéaire ne peut travailler qu'avec des nombres : il faut donc transformer la colonne Location, qui contient du texte (les catégories A, B, C), en valeurs numériques. C'est le rôle de `pd.get_dummies()`, qui crée une colonne binaire (0 ou 1, affichée ici True/False) pour chaque catégorie.

Préparer la colonne "Location"

[3]
✓ 0 s

```
# Transformer le texte de la colonne Location en chiffres
df = pd.get_dummies(df, columns=['Location'], drop_first=True)

# Vérifier le résultat
df.head()
```

	HouseSize	DistanceToCity	Price	Location_B	Location_C
1	48.6	27.90	19.35	True	False
3	65.2	25.99	25.37	False	False
4	47.7	29.19	19.66	False	False
5	47.7	32.02	18.61	False	True
6	65.8	39.43	27.20	True	False

Étapes suivantes : [Générer du code avec df](#) [New interactive sheet](#)

Capture 4 - Cellule [3] : encodage de Location en deux colonnes Location_B et Location_C via `pd.get_dummies(drop_first=True)`.

L'option `drop_first=True` supprime la première catégorie (ici A) pour éviter la redondance entre les colonnes : quand Location_B et Location_C valent toutes les deux False, cela signifie implicitement que la maison est en zone A. C'est une bonne pratique pour éviter le problème de multicollinéarité (colonnes qui se déduisent les unes des autres).

3. Construction du modèle

3.1 Séparation des variables d'entrée (X) et de sortie (y)

La cellule [4] sépare le tableau en deux parties : X regroupe toutes les colonnes explicatives (HouseSize, DistanceToCity, Location_B, Location_C) obtenues en retirant la colonne Price ; y ne contient que la colonne Price, c'est-à-dire la valeur que le modèle doit apprendre à prédire.

Séparer les entrées et la sortie

```
[4]
✓ 0 s
# X contient les variables d'entrées (on enlève juste le prix)
X = df.drop(columns=['Price'])

# y contient la variable de sortie (le prix seul)
y = df['Price']
```

Capture 5 - Cellule [4] : `X = df.drop(columns=['Price'])` et `y = df['Price']`.

3.2 Division en jeu d'entraînement et jeu de test

La cellule [5] utilise `train_test_split` pour séparer les données en deux groupes : 80 % pour entraîner le modèle (`X_train`, `y_train`) et 20 % pour tester ses performances sur des données qu'il n'a jamais vues (`X_test`, `y_test`). Le paramètre `random_state=42` fige le tirage aléatoire, ce qui garantit que le découpage est reproductible d'une exécution à l'autre.

Diviser les données en entraînement et en test

```
[5]
✓ 0 s
# Diviser les données (80% pour l'entraînement, 20% pour le test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Capture 6 — Cellule [5] : `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`.

3.3 Création et entraînement du modèle

La cellule [6] instancie un modèle `LinearRegression()`, puis l'entraîne avec `model.fit(X_train, y_train)`. Pendant cette étape, le modèle calcule les coefficients qui relient chaque variable d'entrée (HouseSize, DistanceToCity, Location_B, Location_C) à la variable de sortie (Price). Le message affiché en sortie confirme que l'entraînement s'est déroulé sans erreur.

Créer et entraîner le modèle

```
[6]
✓ 0 s
# 1. Créer la machine (le modèle de régression linéaire)
model = LinearRegression()

# 2. Entraîner le modèle avec nos données nettoyées
model.fit(X_train, y_train)

print("Le modèle est entraîné avec succès !")

... Le modèle est entraîné avec succès !
```

Capture 7 - Cellule [6] : création du modèle, entraînement avec `model.fit()`, et confirmation « Le modèle est entraîné avec succès ! ».

4. Évaluation du modèle

La dernière cellule [7] mesure la qualité du modèle sur les données de test, qu'il n'a pas vues pendant l'entraînement. Trois étapes : le modèle prédit les prix (`y_pred = model.predict(X_test)`) ; les métriques d'évaluation sont importées (`mean_squared_error`, `r2_score`) ; puis le R^2 et le MSE sont calculés en comparant `y_test` (les vraies valeurs) à `y_pred` (les valeurs prédites).

Calculer les scores du modèle

```
[7] ✓ 0 s # 1. Demander au modèle de prédire les prix des maisons de test
y_pred = model.predict(x_test)

# 2. Importer les outils pour calculer les scores
from sklearn.metrics import mean_squared_error, r2_score

# 3. Calculer les scores
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# 4. Afficher les résultats
print(f"Score R² (Précision) : {r2}")
print(f"Erreur Moyenne (MSE) : {mse}")

... Score R² (Précision) : 0.9700148640277144
Erreur Moyenne (MSE) : 1.7160829446611439
```

Capture 8 - Cellule [7] : calcul et affichage du Score R^2 et de l'Erreur Moyenne (MSE).

Résultats chiffrés

Indicateur	Valeur obtenue
Score R^2 (précision)	$\approx 0,970$
Erreur quadratique moyenne (MSE)	$\approx 1,716$

Interprétation des résultats

Un R^2 de 0,97 signifie que le modèle explique environ 97 % de la variation du prix des maisons à partir des trois variables utilisées. C'est un très bon score : les variables choisies (surface, distance au centre-ville, zone) sont donc fortement liées au prix, et cette relation est bien capturée par une fonction linéaire.

Le MSE de 1,72 correspond à l'erreur quadratique moyenne, exprimée dans l'unité du prix au carré. En prenant sa racine carrée ($\approx 1,31$), on obtient une erreur moyenne d'environ 1,31 sur l'échelle du prix — ce qui est faible comparé aux valeurs de Price observées dans l'aperçu du tableau (entre 18 et 27 environ). Cela confirme que les prédictions du modèle sont, en moyenne, très proches des valeurs réelles.

Conclusion

Le modèle de régression linéaire multiple répond bien à la consigne : il combine HouseSize, DistanceToCity et la variable Location (encodée numériquement) pour prédire Price, avec de très bonnes performances ($R^2 \approx 0,97$, MSE faible). La démarche suivie - import du fichier, nettoyage des données, encodage de la variable catégorielle, séparation entrées/sortie, découpage train/test, entraînement puis évaluation - correspond exactement à la méthodologie standard d'un projet de machine learning supervisé.

FIN